Java Programming Practical 6	
Q1	Design a Java program to simulate a **Smart Traffic Control System** at a city junction. Create an interface named TrafficSignal that declares methods such as displaySignal(), action(), and duration(). Then create three classes named Ready, Go, and Stop that implement the TrafficSignal interface. Each class should define its own behavior: Ready should alert vehicles to slow down and prepare to stop, Go should allow vehicles to move and display a safe driving message, and Stop should instruct vehicles to halt completely. In the main method, demonstrate how a single TrafficSignal reference can be used to switch between Ready, Go, and Stop objects dynamically to simulate the working cycle of a real traffic signal, clearly showing the concept of abstraction and runtime polymorphism using interfaces.
Q2	A company is building a system for smart devices. An interface Device has an abstract method turnOn() and a default method status() that prints "Device is ready." A class Laptop implements Device but overrides only turnOn(), while Smartphone overrides both turnOn() and status(). Write a program to create objects of both classes, call turnOn() and status() on them.
Q3	Design a Java program that demonstrates multiple inheritance using interfaces and default methods. Create two interfaces, Transaction with a default method processTransaction(double amount) and Alert with a default method sendAlert(String message). Implement both interfaces in a class OnlinePurchase. The class should use the default methods from both interfaces and allow the user to process a transaction and send an alert, demonstrating multiple inheritance with interfaces and default methods.
Q4	Write a Java program to demonstrate the use of static methods in interfaces and also show the ambiguity problem that occurs when a class implements multiple interfaces with default methods having the same signature. Create an interface Calculator with a static method add(int a, int b) that returns the sum of two integers. Also create two interfaces that contain default methods with the same signature and implement them in a class to show the ambiguity. Explain how this conflict can be resolved by overriding the default method in the implementing class.